PDF [Summer 2025] Final Set B V2

PDF [Summer 2025] Final Set A V2

PDF Spring_2026_CSE321_Final_Set_B

PDF Spring_2026_CSE321_Final_Set_A

PDF [Spring 25] Final Set B.docx

PDF [Spring 25] Final Set A.docx

solve all these qus. these are the previous final qus. i have already given you the slides for each topic. so correctly solve it

Compare with other AI Liner ChatGPT Claude

Show code

Summer 2025 — Set B

Question 1a: Code Tracing (Fork & Search)

Output

```
Plaintext

1: Current search range: low=0 high=2
2: Current search range: low=2 high=2
3: Current search range: low=0 high=0
4: Value 8 found at index 0
```

Question 1b: PCB Memory Structure

1. **PCB Section:** Heap Section.
2. **Dynamic Behavior:** The heap section expands dynamically upward toward higher memory addresses during runtime when `malloc()` is executed, whereas the text section maintains a fixed size throughout execution.

Question 2a: VSFS File System Structure

1. **Largest Region:** Data Region (Data Blocks).
2. **Design Principle:** The primary objective of a file system is to store user content. Dedicating the vast majority of disk space to data blocks maximizes usable storage capacity.

Question 2b: Disk Space Exhaustion

1. **Responsible Region:** Inode Table (or Inode Bitmap).
2. **Explanation:** Creating a file requires both an inode for metadata storage and data blocks for file content. If all inodes are consumed, no new files can be created despite available free data block space.

Question 2c: UNIX Inode Calculations

- Pointers per block = $\frac{32\ \text{Bytes}}{8\ \text{Bytes}} = 4\ \text{pointers/block}$
- Direct blocks capacity = $12 \times 32\ \text{B} = 384\ \text{B}$
- Double indirect blocks capacity = $16 \times (4^2 \times 32\ \text{B}) = 16 \times 512\ \text{B} = 8192\ \text{B}$
- Triple indirect blocks capacity = $3 \times (4^3 \times 32\ \text{B}) = 3 \times 2048\ \text{B} = 6144\ \text{B}$

- Remaining target size = $20.375\ \text{KB}(20,864\ \text{B}) - (384 + 8192 + 6144) = 6144\ \text{B}$
- Single indirect block capacity = $4 \times 32\ \text{B} = 128\ \text{B}$
- Number of single indirect blocks = $\frac{6144\ \text{B}}{128\ \text{B}} = 48\ \text{blocks}$

1. **Pointers per block:** 4 pointers
2. **Single indirect blocks:** 48 blocks

Question 2d: Hard Links

1. **Inodes used:** 1 inode (all hard links point to the same inode).
2. `links_count` **field:** 3

Question 2e: Inode Address Calculation

- Inode Table start block = Block index 4 (after Superblock, Group Descriptor, Data Bitmap, Inode Bitmap)
- Start Byte offset = $4 \times 16\ \text{KB} = 65,536\ \text{Bytes}$
- Inode 353 offset = $(353 - 1) \times 128\ \text{Bytes} = 45,056\ \text{Bytes}$
- Total Byte Address = $65,536 + 45,056 = 110,592\ \text{Bytes}$
- Sector Address = $\frac{110,592}{1024\ \text{Bytes/sector}} = 108$

1. **Byte Address:** 110,592 Bytes
2. **Sector Address:** Sector 108

Question 2f: File Read Bitmaps

1. **Bitmap modification:** No, `read()` does not modify the data bitmap.
2. **Justification:** Reading reads existing data blocks without allocating new blocks or modifying file structure.

Question 2g: Data Journaling

1. **Blocks written to Journal:** Transaction Begin Block (TxB), Inode ($I[v2]$), Data Bitmap ($B[v2]$), Data Block (Db), and Transaction End Block (TxE).
2. **Process name:** Checkpointing.

Question 2h: Journaling Performance Impact

1. **Journaling type:** Data Journaling.
2. **Reason:** Data journaling writes all payload data twice—once to the journal log and once to its physical disk location.

Question 2i: Metadata Journaling Order

1. **Danger:** It breaks the write-ahead rule for user data by committing metadata before data blocks reach disk.
2. **Inconsistency:** If a crash occurs after metadata commits, the file inode points to unwritten or garbage disk blocks.

Question 2j: High-Concurrence Web Server

1. **Approach:** Multithreading (Thread Pooling).
2. **Execution:** A main thread accepts incoming connections and dispatches requests to worker threads in a shared process space, bypassing process-creation overhead.

Question 2k: Parallelism Model

1. **Parallelism Type:** Task Parallelism.
2. **Requirement:** The programmer must implement synchronization (e.g., semaphores or condition variables) so Thread 2 waits for Thread 1 to finish fetching network data.

Question 2l: Semaphore Deadlock

1. **Outcome when P0 resumes:** P0 blocks waiting for Q, which is held by P1.
2. **Outcome when P1 executes** `wait(S)`**:** A circular wait condition is established, resulting in Deadlock between P0 and P1.

Question 3a: Address Binding Stages

1. **Identical Addresses:** Compile-time or Load-time binding.
2. **Differing Addresses:** Execution-time (Run-time) binding.

Question 3b: Process Paging Allocation

1. **Allocated Process Capacity:** Up to $13 \times 16,384 \text{ Bytes} = 212,992 \text{ Bytes}$.
2. **Internal Fragmentation:** Maximum possible internal fragmentation is less than 16,384 Bytes.

Question 3c: Shared Libraries

1. **Feature:** Shared Pages (Page Table Mapping).
2. **Requirement:** Code must be reentrant (pure/read-only code).

Question 3d: Hierarchical Page Table

1. **Outer Page Table entries:** $2^7 = 128 \text{ entries}$
2. **Pointer destination:** Each entry points to the physical frame base address of an inner page table.

Question 3e: Demand Paging Exceptions

1. **Bit invalid reason:** Page is not loaded into physical RAM or address is out of process bounds.
2. **Triggered Event:** Page Fault exception.

Question 3f: Page Replacement Comparison

1. **Key Difference:** FIFO uses load time/order; Optimal uses future reference patterns.
2. **Performance Reason:** Optimal possesses complete knowledge of upcoming references, preventing eviction of pages needed in the immediate future.

Question 3g: EAT Access Time Calculation

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $227.6375 = (17 + m)(0.9725) + (17 + 2m)(0.0275)$
- $227.6375 = 17 + 1.0275m \implies m = \frac{210.6375}{1.0275} = 205 \text{ ns}$

Main memory access time: 205 ns

Question 3h: LRU Replacement

1. **Swapped out page:** Page 2
2. **Justification:** Page 2 was accessed least recently among physical frames $\{2, 0, 3\}$ (access sequence: 2, 0, 3).

Question 3i: 3-Level Address Translation

Case I: Logical Address 52078

- Binary (16-bit): `1100 1011 0110 1110`
- Indices: $P_1 = 12$, $P_2 = 11$, $P_3 = 6$, Offset $d = 14$
- Translation: 2nd Outer PMT[12] $\rightarrow$ Frame 90 $\rightarrow$ Outer PMT[11] $\rightarrow$ Frame 4 $\rightarrow$ Inner PMT[6] $\rightarrow$ Frame 256.

- Physical Address = $256 \times 16 + 14 = 4096 + 14 = 4110$

Case II: Logical Address 24179

- Binary (16-bit): `0101 1110 0111 0011`
- Indices: $P_1 = 5, P_2 = 14, P_3 = 7$, Offset $d = 3$
- Translation: 2nd Outer PMT[5] $\rightarrow$ Frame 31 $\rightarrow$ Outer PMT[14] $\rightarrow$ Valid bit = 0 (Invalid).
- Physical Address: **invalid**

Summer 2025 — Set A

Question 1a: Code Tracing (Fork & Search)

Output

```
Plaintext

1: Current search range: low=0, high=2
2: Current search range: low=2, high=2
3: Value 33 found at index 2
4: Current search range: low=0, high=0
```

Question 1b: Local Variable Allocation

1. **PCB Section:** Stack Section.
2. **Dynamic Behavior:** Stack frame is pushed when `compute()` is invoked and popped upon return.

Question 2a: VSFS Control Blocks

1. **Next Fields:** Inode Bitmap and Data Bitmap.
2. **Function:** Inode bitmap tracks free/allocated inodes; Data bitmap tracks free/allocated data blocks.

Question 2b: Permission Changes

1. **Field Altered:** Inode metadata (`ctime` /permissions).
2. **Reason:** Changing metadata attributes alters control structure data, not payload data stored in data blocks.

Question 2c: UNIX Inode Calculations

- Pointers per block = $\frac{128\ \text{B}}{32\ \text{B}} = 4\ \text{pointers/block}$
- Direct blocks capacity = $24 \times 128\ \text{B} = 3072\ \text{B}$
- Single indirect capacity = $40 \times (4 \times 128\ \text{B}) = 20,480\ \text{B}$
- Triple indirect capacity = $5 \times (4^3 \times 128\ \text{B}) = 40,960\ \text{B}$
- Remaining size needed = $87\ \text{KB}(89,088\ \text{B}) - (3072 + 20480 + 40960) = 24,576\ \text{B}$
- Double indirect block capacity = $4^2 \times 128\ \text{B} = 2048\ \text{B}$
- Double indirect blocks = $\frac{24{,}576\ \text{B}}{2048\ \text{B}} = 12\ \text{blocks}$

1. **Pointers per block:** 4 pointers
2. **Double indirect blocks:** 12 blocks

Question 2d: Deleting Last Hard Link

1. **Freed Resources:** File Inode and allocated Data Blocks.

2. **Updated Fields:** Inode Bitmap, Data Bitmap, and Directory Entries.

Question 2e: Inode Address Calculation

- Inode Table start block = Block 5 ($5 \times 8\ \text{KB} = 40\ \text{KB}$)
- Inode 19 offset = $(19 - 1) \times 1\ \text{KB} = 18\ \text{KB}$
- Total Address = $40\ \text{KB} + 18\ \text{KB} = 58\ \text{KB} = 59,392\ \text{Bytes}$
- Sector Address = $\frac{59,392}{512\ \text{Bytes/sector}} = 116$

1. **Byte Address:** 59,392 Bytes
2. **Sector Address:** Sector 116

Question 2f: File Creation Writes

1. Update Inode Bitmap (allocate new inode).
2. Write new file Inode.
3. Update parent directory `/tmp` Data Block (add file-inode entry).
4. Update parent directory `/tmp` Inode (modification timestamp and size).

Question 2g: Reordered Journal Writes

1. **Danger:** Disk reordering can commit TxE (completion marker) before transaction blocks hit persistent storage.
2. **Incorrect Data:** Replaying an incomplete transaction writes uninitialized/corrupted data during crash recovery.

Question 2h: Journaling Ordering Rules

1. **Difference:** Data journaling logs payload data and metadata; metadata journaling logs metadata only.
2. **Ordering Rule:** User data blocks must be written to disk before the associated metadata transaction commits in the journal.

Question 2i: Directory Block Reuse Crashes

1. **Issue:** Replaying the journal entry writes old directory content over the newly created file's data block.
2. **Prevention Record:** Revoke Records.

Question 2j: Concurrent Web Browsing

1. **Feature:** Multithreading.
2. **Application:** One thread fetches background images while a secondary UI thread handles text rendering and scrolling.

Question 2k: Data Parallelism

1. **Type:** Data Parallelism.
2. **Characteristic:** Identical computation logic executes simultaneously across distinct subsets of data on separate processor cores.

Question 2l: Semaphore Waiting Queues

1. **Potential Issue:** Deadlock.
2. **Specific Event:** Signal operation (`signal()` / `V()`) on a semaphore.

Question 3a: Memory Translation Hardware

1. **Addresses:** Logical Address (`0x400`) and Physical Address (`0x18400`).
2. **Hardware Unit:** Memory Management Unit (MMU).

Question 3b: Process Page Sizing

- Process size = 145,698 Bytes across 18 pages.
- Standard Frame Size = $8192\ \text{Bytes}$ ($8\ \text{KB}$), since $17 \times 8192 < 145,698 \leq 18 \times 8192$.
- Allocation Size = $18 \times 8192 = 147,456\ \text{Bytes}$
- Internal Fragmentation = $147,456 - 145,698 = 1758\ \text{Bytes}$

1. **Page Size:** 8192 Bytes (8 KB)
2. **Internal Fragmentation:** 1758 Bytes

Question 3c: Out-of-Bounds Page Access

1. **Attribute:** Valid-Invalid Bit (V/I bit).
2. **Mechanism:** The OS marks process-mapped pages as "valid" and unmapped/illegal memory pages as "invalid", raising a trap on illegal accesses.

Question 3d: 2-Level Page Table Size

1. **Inner Page Table Entries:** $2^9 = 512\ \text{entries}$
2. **Pointer Destination:** Each entry points to a physical frame number in RAM.

Question 3e: Demand Paging Startup

1. **Technique:** Pure Demand Paging.
2. **Consequence:** Page Fault exception.

Question 3f: FIFO Page Replacement

1. **Replaced Page:** Page P1.
2. **Data Structure:** First-In, First-Out Queue.

Question 3g: TLB Miss Access Time Calculation

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $142.3125 = (\text{TLB} + 125)(0.9575) + (\text{TLB} + 250)(0.0425)$
- $142.3125 = \text{TLB} + 130.3125 \implies \text{TLB} = 12\ \text{ns}$

TLB access time: 12 ns

Question 3h: Optimal Page Replacement

1. **Swapped out page:** Page 1
2. **Justification:** Page 0 is accessed immediately, Page 7 is accessed later, and Page 1 is never referenced again in the future stream.

Question 3i: 3-Level Address Translation

Case I: Logical Address 22135

- Binary (16-bit): `0101 0110 0111 0111`
- Indices: $P_1 = 1$, $P_2 = 5$, $P_3 = 9$, Offset $d = 55$
- Translation: 2nd Outer PMT[1] $\rightarrow$ Frame 27 $\rightarrow$ Outer PMT[5] $\rightarrow$ Frame 2 $\rightarrow$ Inner PMT[9] $\rightarrow$ Frame 125.
- Physical Address = $125 \times 64 + 55 = 8000 + 55 = 8055$

Case II: Logical Address 41046

- Binary (16-bit): `1010 0000 0101 0110`
- Indices: $P_1 = 2$, $P_2 = 0$, $P_3 = 5$, Offset $d = 22$

- Translation: 2nd Outer PMT[2] $\rightarrow$ Frame 13 $\rightarrow$ Outer PMT[0] $\rightarrow$ Valid bit = 0 (Invalid).
- Physical Address: **invalid**

Spring 2026 — Set B

Question 1: Process Interrupt Context
(i) **Selected Item:** (iv) Process B's open file table
(ii) **Explanation:** Open file tables represent kernel-managed file state rather than hardware register context. Hardware registers (SP, PC, general-purpose) are saved to and restored from the PCB during context switches, while open file references may be altered asynchronously by shared system handlers.

Question 2a: Counting Semaphore Misuse
(i) **Incorrect Operation:** Executing `signal()` extra times (or missing `wait()`).
(ii) **Reason:** Executing extra `signal()` calls increments the counting semaphore past its initial bound N, enabling more than N processes to access N resources simultaneously.

Question 2b: Producer-Consumer Deadlock Fix
(i) **Issue:** Deadlock occurs because `wait(mutex)` precedes `wait(empty)` in Producer and `wait(full)` in Consumer. If the buffer fills, the Producer acquires `mutex` and blocks on `empty`, locking out the Consumer from acquiring `mutex` to drain items.
(ii) **Fix:** Move `wait(empty)` before `wait(mutex)` in Producer, and move `wait(full)` before `wait(mutex)` in Consumer.

Question 2c: Concurrent Page Allocation Fix
(i) **Issue:** A counting semaphore `page_sem = N` allows up to N threads into `allocate_page()` simultaneously. They race on the un-synchronized `page_table` array, leading to duplicate page allocations and corrupted state.
(ii) **Fix:** Enclose the `for` loop scan and array update inside a critical section guarded by a binary mutex semaphore (`sem_t mutex = 1;`).

Question 3a: FS Block Calculations
(i) **Data blocks:** $\frac{1\text{ MB}}{256\text{ B}} = 4096\text{ blocks}$
(ii) **Data block bitmap size:** $\frac{4096\text{ bits}}{8\text{ bits/byte}} = 512\text{ Bytes} = 2\text{ blocks}$
(iii) **Inode blocks:** $\lceil \frac{65 \times 32\text{ B}}{256\text{ B}} \rceil = \lceil \frac{2080}{256} \rceil = 9\text{ blocks}$
(iv) **Inode Bitmap internal fragmentation:** $256\text{ Bytes} - \lceil \frac{65}{8} \rceil\text{ Bytes} = 256 - 9 = 247\text{ Bytes}$

Question 3b: UNIX Inode Maximum File Size

- Block size = 2048 B, Pointer size = 2 B $\implies$ 1024 pointers/block.
- Direct limit = $12 \times 2\text{ KB} = 24\text{ KB}$
- Single indirect limit = $1024 \times 2\text{ KB} = 2\text{ MB}$
- Double indirect limit = $1024^2 \times 2\text{ KB} = 2\text{ GB}$
- Triple indirect limit = $1024^3 \times 2\text{ KB} = 2\text{ TB}$
- Maximum capacity = $2\text{ TB} + 2\text{ GB} + 2\text{ MB} + 24\text{ KB} \approx 2.002\text{ TB}$.

Conclusion: No, the file system cannot store a 3.8 TB file because its hard upper limit is ~2.002 TB.

Question 3c: Inode Crash Inconsistency
(i) **Inconsistency:** The data block bitmap marks the new block allocated and payload data is written, but the file inode lacks a pointer reference (Block Leak).
(ii) `cat` **output:** The appended log lines do not appear; file size and contents remain at their pre-append state.
(iii) **Impact:** Storage capacity leaks progressively over time as unreachable data blocks remain marked occupied.

Question 3d: Directory Block Reclamation
The file system reclaims a directory block when all directory entries within that specific block are

deleted, dropping its active entry count to zero.

Question 3e: Inode Address Calculation

- Inode Table start byte = $5 \text{ blocks} \times 1024 \text{ B} = 5120 \text{ Bytes}$
- Inode 238 address = $5120 + (238 - 1) \times 128 = 5120 + 30,336 = 35,456 \text{ Bytes}$
- Sector Address = $\lfloor \frac{35,456}{2048 \text{ B/sector}} \rfloor = 17$

(i) **Byte Address:** 35,456 Bytes
(ii) **Sector Address:** Sector 17

Question 4a: Single-Level Page Table Sizing

- Logical pages = $\frac{2^{40}}{2^{14}} = 2^{26} \text{ pages}$
- Physical frames = $\frac{2^{35}}{2^{14}} = 2^{21} \text{ frames} \implies 21 \text{ bits}$
- PTE size = $21 + 3 = 24 \text{ bits} = 3 \text{ Bytes}$
- Page Table Size = $2^{26} \times 3 \text{ Bytes} = 201,326,592 \text{ Bytes} = 192 \text{ MB}$

Question 4b: Hierarchical Page Table Breakdown

- Logical address = 40 bits, Offset = 11 bits ($2^{11} = 2 \text{ KB}$).
- Page Number bits = $40 - 11 = 29 \text{ bits}$.
- Entries per page = $\frac{2048 \text{ B}}{2 \text{ B}} = 1024 \text{ entries} \implies 10 \text{ bits/level}$.
- Structure: P_1 (Top level outer): 9 bits, P_2 (Middle level outer): 10 bits, P_3 (Inner level): 10 bits, Offset d: 11 bits.

Question 4c: 5-Level Paging EAT

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 6m) \cdot \text{miss}$
- $\text{EAT} = (20 + 150)(0.90) + (20 + 6 \times 150)(0.10)$
- $\text{EAT} = 170(0.90) + 920(0.10) = 153 + 92 = 245 \text{ ns}$

Question 4d: LRU Page Replacement
Sequence: `1, 3, 1, 2, 4, 6, 2, 4, 6, 1` (3 frames)

Step	Access	Memory State	Status
1	1	[1, _, _]	Fault
2	3	[1, 3, _]	Fault
3	1	[1, 3, _]	Hit
4	2	[1, 3, 2]	Fault
5	4	[1, 4, 2]	Fault (Replace
6	6	[6, 4, 2]	Fault (Replace
7	2	[6, 4, 2]	Hit
8	4	[6, 4, 2]	Hit
9	6	[6, 4, 2]	Hit

Step	Access	Memory State	Status
10	1	[6, 4, 1]	Fault (Replace

...

(i) Simulation steps as shown above.
(ii) **Hit Ratio:** $\frac{4}{10} = 0.40 (40\%)$

Spring 2026 — Set A

Question 1: Process Interrupt Context
(i) **Selected Item:** (v) Page Table Base Register
(ii) **Explanation:** Context switches restore CPU general registers, stack pointers, and execution pointers directly. The PTBR is re-loaded to point to Process A's root page directory upon resumption, whereas inner page table mapping dynamic entries can be altered while dormant.

Question 2a: Semaphore Device Starvation
(i) **Incorrect Operation:** Failure to execute `signal()` (or executing extra `wait()` calls).
(ii) **Reason:** Processes exit without calling `signal()`, permanently depleting permits and reducing available allocation capacity below N.

Question 2b: Producer-Consumer Initialization Fix
(i) **Issue:** `empty` and `full` are incorrectly initialized to $N/2$. If buffer space is N, `empty` must start at N and `full` at 0.
(ii) **Fix:** Set line 2 to `empty = N` and line 3 to `full = 0`. Swap `wait` ordering so resource checks occur before acquiring `mutex`.

Question 2c: Toll Booth Race Condition
(i) **Issue:** Initializing `booth_sem = N` allows N threads to run `assign_booth()` simultaneously, creating a race condition over `booth_status` elements.
(ii) **Fix:** Wrap the array scanning logic with a binary mutex semaphore `sem_t mutex = 1;`.

Question 3a: Miniature FS Sizing
(i) **Data blocks:** $\frac{1\ \text{MB}}{256\ \text{B}} = 4096\ \text{blocks}$
(ii) **Data block bitmap size:** $\frac{4096\ \text{bits}}{8} = 512\ \text{Bytes} = 2\ \text{blocks}$
(iii) **Inode blocks:** $\lceil \frac{65 \times 32}{256} \rceil = 9\ \text{blocks}$
(iv) **Inode Bitmap internal fragmentation:** $256 - 9 = 247\ \text{Bytes}$

Question 3b: UNIX Inode File Capacity Validation

- Block size = 4096 B, Pointer size = 4 B $\implies$ 1024 pointers/block.
- Direct limit = $12 \times 4\ \text{KB} = 48\ \text{KB}$
- Single indirect limit = $1024 \times 4\ \text{KB} = 4\ \text{MB}$
- Double indirect limit = $1024^2 \times 4\ \text{KB} = 4\ \text{GB}$
- Triple indirect limit = $1024^3 \times 4\ \text{KB} = 4\ \text{TB}$
- Maximum file capacity $\approx 4\ \text{TB} + 4\ \text{GB} + 4\ \text{MB} + 48\ \text{KB} = 4.004\ \text{TB}$.

Validation: Yes, the file system can store a 3.9 TB file because 3.9 TB is within the maximum limit of 4.004 TB.

Question 3c: Non-Journaling Crash Analysis
(i) **Inconsistency:** Data bitmap marks block allocated and contents are written, but inode pointers remain un-updated.
(ii) `cat` **output:** File contents are unchanged (new data invisible).
(iii) **Storage Impact:** Leaked storage space accumulates over time.

Question 3d: Directory Reclamation
A directory data block is reclaimed when all internal directory entry structures in that block are deleted.

Question 3e: Inode Address Calculation

- Inode Table start byte = $7 \times 2048 = 14,336 \text{ Bytes}$
- Inode 513 address = $14,336 + (513 - 1) \times 512 = 14,336 + 262,144 = 276,480 \text{ Bytes}$
- Sector Address = $\frac{276,480}{1024} = 270$

(i) **Byte Address:** 276,480 Bytes
(ii) **Sector Address:** Sector 270

Question 4a: Single-Level Page Table Size

- Pages = $\frac{2^{32}}{2^{12}} = 2^{20} \text{ pages}$
- Frames = $\frac{2^{31}}{2^{12}} = 2^{19} \text{ frames} \implies 19 \text{ bits}$
- PTE size = $19 + 13 = 32 \text{ bits} = 4 \text{ Bytes}$
- Page Table size = $2^{20} \times 4 \text{ Bytes} = 4 \text{ MB}$

Question 4b: Hierarchical Paging Design

- Logical address = 18 bits, Offset = 8 bits ($2^8 = 256$ B).
- Page number bits = 10 bits. PTE size = 1 Byte $\implies 256$ entries/page (8 bits/level).
- Structure: P_1 (Outer level): 2 bits, P_2 (Inner level): 8 bits, Offset d: 8 bits.

Question 4c: 4-Level Paging EAT

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 5m) \cdot \text{miss}$
- $\text{EAT} = (10 + 100)(0.95) + (10 + 5 \times 100)(0.05)$
- $\text{EAT} = 110(0.95) + 510(0.05) = 104.5 + 25.5 = 130 \text{ ns}$

Question 4d: OPT Page Replacement
Sequence: `1, 6, 3, 1, 5, 4, 5, 6, 1, 5` (3 frames)

Step	Access	Memory State	Status
1	1	[1, _, _]	Fault
2	6	[1, 6, _]	Fault
3	3	[1, 6, 3]	Fault
4	1	[1, 6, 3]	Hit
5	5	[1, 6, 5]	Fault (Replace
6	4	[1, 4, 5]	Fault (Replace
7	5	[1, 4, 5]	Hit
8	6	[1, 6, 5]	Fault (Replace
9	1	[1, 6, 5]	Hit
10	5	[1, 6, 5]	Hit

...

(i) Simulation steps as shown above.
(ii) **Hit Ratio:** $\frac{4}{10} = 0.40 (40\%)$

Spring 2025 — Set B

Question 1a: Dining Philosophers Error
Line 4 executes `wait(chopstick[(i++) % 5])`, which increments `i`. Lines 7–8 release chopsticks using modified `i` indices, causing improper semaphore releases and unlocking errors.

Question 1b: Deadlock Order
Method 2 gets stuck first because it acquires `mutex` and then waits on `sem2` (initialized to 0), holding `mutex` indefinitely while Method 1 waits for `mutex`.

Question 2a: Hardware Translation Cache
Translation Lookaside Buffer (TLB).

Question 2b: Main Memory Access Time Calculation

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $293 = (23 + m)(0.955) + (23 + 2m)(0.045)$
- $293 = 23 + 1.045m \implies m = \frac{270}{1.045} \approx 258.37 \text{ ns}$

Question 2c: Paging Physical Mapping

- Main memory = 64 B, Page size = 8 B $\implies$ 8 physical frames (frames 0–7). Offset = 3 bits.

1. **Address 13:** Page 1, Offset 5 $\rightarrow$ Frame 0 $\rightarrow$ Physical Address = $0 \times 8 + 5 = 5$
2. **Address 26:** Page 3, Offset 2 $\rightarrow$ Frame 15 (Out of physical bounds 0–7) $\rightarrow$ **invalid**
3. **Address 4:** Page 0, Offset 4 $\rightarrow$ Frame 5 $\rightarrow$ Physical Address = $5 \times 8 + 4 = 44$
4. **Address 22:** Page 2, Offset 6 $\rightarrow$ Frame 7 $\rightarrow$ Physical Address = $7 \times 8 + 6 = 62$

User View Frame Mapping:

- Frame 0: `touch` (Page 1)
- Frame 5: `chmod` (Page 0)
- Frame 7: `rm` (Page 2)

Question 2d: 64-Bit Address Multilevel Paging

- Page size = 4 KB (2^{12} B) $\implies$ Offset = 12 bits.
- Page number bits = $64 - 12 = 52$ bits.
- PTE size = 1 Byte $\implies$ $\frac{4096}{1} = 4096 \text{ entries/frame} \implies 12 \text{ bits/level}$.
- Reduction steps: $52 \rightarrow 40 \rightarrow 28 \rightarrow 16 \rightarrow 4$ bits.
- 5th Level (Outermost) table contains $2^4 = 16 \text{ entries} \times 1 \text{ B} = 16 \text{ Bytes} \leq 4096 \text{ Bytes}$ (Fits in 1 frame).

Address Structure: 5-Level Paging (P_1: 4 bits, P_2: 12 bits, P_3: 12 bits, P_4: 12 bits, P_5: 12 bits, Offset d: 12 bits).

Question 2e: Buggy Optimal Page Replacement
Sequence: `7, 3, 2, 1, 2, 3, 7, 8, 3, 5, 8, 2` (4 frames)
Rule: Replaces optimal target AND adjacent next frame.

Ref	Memory State	Status
7	[7, _, _, _]	Fault

Ref	Memory State	Status
3	[7, 3, _, _]	Fault
2	[7, 3, 2, _]	Fault
1	[7, 3, 2, 1]	Fault
2	[7, 3, 2, 1]	Hit
3	[7, 3, 2, 1]	Hit
7	[7, 3, 2, 1]	Hit
8	[8, _, 2, 1]	Fault (Replaces frame 0 target 7 & frame 1 page 3)
3	[8, 3, 2, 1]	Fault
5	[8, 5, _, 1]	Fault (Replaces frame 1 target 3 & frame 2 page 2)
8	[8, 5, _, 1]	Hit
2	[8, 5, 2, 1]	Fault

...

- Total Faults = 8, Total Hits = 4
- **Hit Ratio:** $\frac{4}{12} = \frac{1}{3}(33.33\%)$

Question 3a: Post-Crash State Deduction

- **Data block write:** Successful
- **Data bitmap update:** Failed
- **Inode update:** Successful

Question 3b: UNIX Inode Calculation

- Block size = 64 B, Pointer size = 8 B $\implies$ 8 pointers/block.
- Direct limit = $12 \times 64 = 768\ \text{B}$
- 3 Single indirect limit = $3 \times (8 \times 64) = 1536\ \text{B}$
- 2 Triple indirect limit = $2 \times (8^3 \times 64) = 65,536\ \text{B}$
- Remaining size = $82.25\ \text{KB}(84,224\ \text{B}) - (768 + 1536 + 65536) = 16,384\ \text{Bytes}$
- Double indirect block capacity = $8^2 \times 64 = 4096\ \text{Bytes}$
- Double indirect blocks = $\frac{16,384}{4096} = 4\ \text{blocks}$

Question 3c: Inode Sector Calculation

- Start Byte = $5 \times 2048 + (28 - 1) \times 256 = 10,240 + 6912 = 17,152\ \text{Bytes}$
- Sector Address = $\frac{17,152}{256} = 67$

Sector Address: Sector 67

Question 3d: File Access Timeline (`/nf/h`)

1. `open("/nf/h")` : Read root `/` inode → locate `nf` directory → read `nf` inode → locate `h` file → read `h` inode.
2. `write()` : Scan data bitmap → allocate 2 free data blocks → write content to new data blocks → update `h` inode (pointers, size, mtime) → flush inode to disk.

Question 4: Protection Access Matrix
Granting `Write*` to domain D_2 over object F_1 updates entry (D_2, F_1) to include `Read*` , `Write*` .

Spring 2025 — Set A

Question 1a: Code Error
Line 7 executes `signal(chopstick[++i])` , which pre-increments `i` , corrupting the variable and releasing an unintended chopstick index.

Question 1b: Deadlock Order
`func1` gets stuck first because it acquires `mutex1` and attempts to wait on `mutex2` , creating a circular wait condition with `func2` .

Question 2a: Shared Code Paging
Shared Pages (Reentrant Code).

Question 2b: TLB Miss Access Time

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $350 = (\text{TLB} + 250)(0.98) + (\text{TLB} + 500)(0.02)$
- $350 = \text{TLB} + 255 \implies \text{TLB} = 95 \text{ ns}$

Question 2c: Paging Physical Mapping

- Main memory = 128 B, Page size = 16 B $\implies$ 8 physical frames (0–7).

1. **Address 54:** Page 3, Offset 6 → Frame 1 → Physical Address = $1 \times 16 + 6 = 22$
2. **Address 9:** Page 0, Offset 9 → Frame 3 → Physical Address = $3 \times 16 + 9 = 57$
3. **Address 30:** Page 1, Offset 14 → Frame 5 → Physical Address = $5 \times 16 + 14 = 94$
4. **Address 35:** Page 2, Offset 3 → Frame 9 (Out of bounds 0–7) → **invalid**

User View Frame Mapping:

- Frame 1: `mkdir` (Page 3)
- Frame 3: `man` (Page 0)
- Frame 5: `uname` (Page 1)

Question 2d: 64-Bit Address Multilevel Paging

- Page size = 8 KB (2^{13} B) $\implies$ Offset = 13 bits. Page number bits = $64 - 13 = 51$ bits.
- PTE size = 2 Bytes $\implies \frac{8192}{2} = 4096 \text{ entries/frame} \implies 12 \text{ bits/level}$.
- Reduction steps: $51 \to 39 \to 27 \to 15 \to 3$ bits.
- 5th Level (Outermost) table size = $2^3 = 8 \text{ entries} \times 2 \text{ B} = 16 \text{ Bytes} \leq 8192 \text{ Bytes}$.

Address Structure: 5-Level Paging (P_1: 3 bits, P_2: 12 bits, P_3: 12 bits, P_4: 12 bits, P_5: 12 bits, Offset d: 13 bits).

Question 2e: Buggy LRU Page Replacement
Sequence: `1, 7, 6, 1, 3, 5, 7, 2, 9, 8, 1, 9` (4 frames)
Rule: Evicts LRU target AND next adjacent frame.

Ref	Memory State	Status
1	[1, _, _, _]	Fault
7	[1, 7, _, _]	Fault
6	[1, 7, 6, _]	Fault
1	[1, 7, 6, _]	Hit
3	[1, 7, 6, 3]	Fault
5	[1, 5, _, 3]	Fault (Replaces frame 1 target 7 & frame 2 page 6)
7	[1, 5, 7, 3]	Fault
2	[2, _, 7, 3]	Fault (Replaces frame 0 target 1 & frame 1 page 5)
9	[2, 9, 7, 3]	Fault
8	[2, 9, 8, _]	Fault (Replaces frame 2 target 7 & frame 3 page 3)
1	[2, 9, 8, 1]	Fault
9	[2, 9, 8, 1]	Hit

...

- Faults = 10, Total Refs = 12
- **Page Fault Ratio:** $\frac{10}{12} = \frac{5}{6} (83.33\%)$

Question 3a: Post-Crash State Deduction

- **Data block write:** Failed
- **Data bitmap update:** Failed
- **Inode update:** Failed

Question 3b: UNIX Inode Calculation

- Block size = 128 B, Pointer size = 8 B $\implies$ 16 pointers/block.
- Direct limit = $10 \times 128 = 1280 \text{ B}$
- 2 Single indirect limit = $2 \times (16 \times 128) = 4096 \text{ B}$
- 2 Double indirect limit = $2 \times (16^2 \times 128) = 65,536 \text{ B}$
- Remaining size = $1093.25 \text{ KB} (1,119,488 \text{ B}) - (1280 + 4096 + 65536) = 1,048,576 \text{ Bytes}$
- Triple indirect capacity per block = $16^3 \times 128 = 524,288 \text{ Bytes}$
- Triple indirect blocks = $\frac{1,048,576}{524,288} = 2 \text{ blocks}$

Question 3c: Inode Address Calculation

- Address = $4 \times 4096 + (84 - 1) \times 128 = 16,384 + 10,624 = 27,008 \text{ Bytes}$

- Sector Address = $\frac{27,008}{1024} = 26$

Sector Address: Sector 26

Question 3d: Reading Specific File Blocks

1. `open("/f1/f2/a")`: Traverse `/` $\rightarrow$ `f1` $\rightarrow$ `f2` $\rightarrow$ read file `a` inode.
2. `read()`: Read 2nd direct block pointer $\rightarrow$ fetch 2nd data block; read 3rd direct block pointer $\rightarrow$ fetch 3rd data block.

Question 4: Access Matrix Transfer
In domain D_2, adding ownership right transfers `Owner` permission to (D_2, F_1).

Fall 2024 — Set A

Question 1: Parallel Efficiency & Thread Model

1. **Performance Change Calculation (Amdahl's Law):**
    1. $S = \frac{1}{(1-P)+\frac{P}{N}}$ with $P = 0.68$.
    2. Speedup at $N = 7$: $S_7 = \frac{1}{0.32+\frac{0.68}{7}} \approx 2.397$
    3. Speedup at $N = 9$: $S_9 = \frac{1}{0.32+\frac{0.68}{9}} \approx 2.528$
    4. Performance Improvement: $\frac{2.528-2.397}{2.397} \times 100\% = 5.46\%$ increase.
2. **Multithreading Model: Many-to-One Model.**
    1. **Drawbacks:** The entire process blocks if one thread makes a blocking system call, and execution cannot scale across multiple CPU cores in parallel.

Question 2a: Critical Section Locking

1. **Method:** Mutex Lock.
2. **Solved Problems:** Prevents Race Conditions, Data Inconsistency, and Mutual Exclusion violations.

Question 2b: Semaphore Code Tracing

Output

```
Plaintext

Current Value of Shared: 50
Current Value of Shared: 53
Current Value of Shared: 56
Current Value of Shared: 59
Current Value of Shared: 54
Final Value of Shared: 49
```

Question 2c: Peterson's Solution Execution Table

Time Window	Active Process	Statement Executed	Statement Typ
0–4 ms	Process 1	`flag[0] = true`	Assignment
4–8 ms	Process 1	`turn[1] = true`	Assignment

Time Window	Active Process	Statement Executed	Statement Typ
8–10 ms	Process 1	`while(flag[1] == true...)`	Logic
10–12 ms	Process 1	Pre-critical entry	Execution
12–16 ms	Process 0	`flag[1] = true`	Assignment
16–20 ms	Process 0	`turn[0] = true`	Assignment
20–22 ms	Process 0	`while(flag[0] == true...)`	Logic

...

Question 3a: Dynamic Partitioning

- Available Holes: 200K, 260K, 160K, 250K, 400K, 50K
- Demands: P1 (100K), P2 (300K), P3 (200K), P4 (145K), P5 (150K), P6 (60K), P7 (50K)

Worst-Fit Allocation:

- P1 (100K) $\rightarrow$ 400K hole (leaves 300K)
- P2 (300K) $\rightarrow$ 300K hole (leaves 0K)
- P3 (200K) $\rightarrow$ 260K hole (leaves 60K)
- P4 (145K) $\rightarrow$ 250K hole (leaves 105K)
- P5 (150K) $\rightarrow$ 200K hole (leaves 50K)
- P6 (60K) $\rightarrow$ 160K hole (leaves 100K)
- P7 (50K) $\rightarrow$ 105K hole (leaves 55K)
- **External Fragmentation:** 0 KB

First-Fit Allocation:

- P1 (100K) $\rightarrow$ 200K hole (leaves 100K)
- P2 (300K) $\rightarrow$ 400K hole (leaves 100K)
- P3 (200K) $\rightarrow$ 260K hole (leaves 60K)
- P4 (145K) $\rightarrow$ 160K hole (leaves 15K)
- P5 (150K) $\rightarrow$ 250K hole (leaves 100K)
- P6 (60K) $\rightarrow$ 100K (former 200K) hole (leaves 40K)
- P7 (50K) $\rightarrow$ 100K (former 400K) hole (leaves 50K)
- **External Fragmentation:** 0 KB

Comparison: Both algorithms accommodate all processes, but Worst-Fit preserves larger remaining contiguous blocks.

Question 3b: TLB Miss Access Time

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $253 = (\text{TLB} + 150)(0.85) + (\text{TLB} + 300)(0.15)$
- $253 = \text{TLB} + 172.5 \implies \text{TLB} = 80.5 \text{ ns}$

Question 4a: Physical Address Translation

- Main Memory = 64 B, Page Size = 8 B $\implies$ 8 frames.

1. **Address 21:** Page 2, Offset 5 $\rightarrow$ Frame 3 $\rightarrow$ Physical Address = $3 \times 8 + 5 = 29$
2. **Address 36:** Page 4, Offset 4 $\rightarrow$ Frame 7 $\rightarrow$ Physical Address = $7 \times 8 + 4 = 60$
3. **Address 3:** Page 0, Offset 3 $\rightarrow$ Frame 6 $\rightarrow$ Physical Address = $6 \times 8 + 3 = 51$
4. **Address 30:** Page 3, Offset 6 $\rightarrow$ Frame 8 (Out of bounds) $\rightarrow$ **invalid**
5. **Address 41:** Page 5, Offset 1 $\rightarrow$ Frame 12 (Out of bounds) $\rightarrow$ **invalid**
6. **Address 15:** Page 1, Offset 7 $\rightarrow$ Frame 1 $\rightarrow$ Physical Address = $1 \times 8 + 7 = 15$

Question 4b: LRU vs. Sub-Optimal Algorithm
Sequence: `7, 4, 6, 7, 2, 0, 5, 1, 4, 7, 1, 2` (3 frames)

- **Standard LRU Performance:** 10 Page Faults (Fault Ratio = 83.33%)
- **Sub-Optimal Algorithm Performance:** 11 Page Faults (Fault Ratio = 91.67%)
- **Comparison:** Standard LRU performs better because the sub-optimal variant replaces pages that are needed in the nearest future, increasing overall page faults.

Fall 2025 — Set B

Question 1a: File System Write Locations
Writes take place in the Inode Bitmap, Data Bitmap, Parent Directory Data Block (`/CSE321/Fall2025/Questions`), Parent Directory Inode, and newly allocated File Inode.

Question 1b: Journal Crash Recovery
(i) **Issue:** Transaction `id=52` includes an Inode update $I(F9)$ but lacks a matching TxE (commit) marker, leaving an uncommitted transaction.
(ii) **Prevention:** The file system discards uncommitted transactions during journal recovery.

Question 1c: Indirect Inode Disk Reads

- Block size = 32 B, Pointer size = 4 B $\implies$ 8 pointers/block.
- Direct limit = $6 \times 32 = 192\ \text{Bytes}$.
- The 240th Byte falls into the Single Indirect block range (Offset $240 - 192 = 48$ B).
- **Required Reads:** Inode read $\rightarrow$ Single Indirect Block read $\rightarrow$ Target Data Block read (3 Disk Reads).

Question 1d: Inode Address Calculation

- Start Byte = $7 \times 1024 + (27 - 1) \times 128 = 7168 + 3328 = 10,496\ \text{Bytes}$
- Sector Address = $\frac{10,496}{1024} = 10$

(i) **Byte Address:** 10,496 Bytes
(ii) **Sector Address:** Sector 10

Question 1e: Small Journal Performance Impact
Allocating a small journal forces frequent checkpointing flushes to main storage, creating severe I/O bottlenecks.

Question 2a: Internal Fragmentation in Paging
No, paging cannot completely eliminate internal fragmentation because process memory is allocated in fixed page blocks. The severity is reduced by implementing smaller page sizes.

Question 2b: Page Table Size Calculation

- Pages = $\frac{2^{48}}{2^{17}} = 2^{31}\ \text{pages}$
- Physical Frames (4 GB RAM) = $\frac{2^{32}}{2^{17}} = 2^{15}\ \text{frames} \implies 15\ \text{bits}$
- PTE size = $15 + 3 = 18\ \text{bits} \implies 3\ \text{Bytes}$
- Page Table Size = $2^{31} \times 3\ \text{Bytes} = 6\ \text{GB}$

Question 2c: Copy-On-Write Page Allocation

- Array size = $1400 \times 4\ \text{Bytes} = 5600\ \text{Bytes}$. Page size = 2048 Bytes $\implies$ 3 pages.
  (i) **Minimum newly created pages:** 3 pages
  (ii) **Maximum newly created pages:** 6 pages

Question 2d: TLB Miss EAT Calculation

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $390.275 = (24 + m)(0.9535) + (24 + 2m)(0.0465)$
- $390.275 = 24 + 1.0465m \implies m = 350\ \text{ns}$

Main memory access time: 350 ns

Question 2e: Non-Contiguous Frame Allocation
Yes, the process can be created because paging natively supports non-contiguous physical frame allocation in RAM.

Question 2f: 3-Level Address Translation

Case i: Logical Address 11059

- Binary (16-bit): `0010 1011 0011 0011`
- Indices: $P_1 = 0$, $P_2 = 10$, $P_3 = 12$, Offset $d = 51$
- Translation: 2nd Outer PMT[0] $\rightarrow$ Frame 50 $\rightarrow$ Outer PMT[10] $\rightarrow$ Frame 2 $\rightarrow$ Inner PMT[12] $\rightarrow$ Frame 45.
- Physical Address = $45 \times 64 + 51 = 2880 + 51 = 2931$

Case ii: Logical Address 47321

- Binary (16-bit): `1011 1000 1101 1001`
- Indices: $P_1 = 2$, $P_2 = 14$, $P_3 = 13$, Offset $d = 25$
- Translation: 2nd Outer PMT[2] $\rightarrow$ Frame 23 $\rightarrow$ Outer PMT[14] $\rightarrow$ Frame 7 $\rightarrow$ Inner PMT[13] $\rightarrow$ Invalid bit.
- Physical Address: **invalid**

Question 3a: Dynamic Domain Expansion
Use domain switching mechanisms (such as `setuid` or dynamic rights amplification) to transition execution into a higher-privileged domain.

Question 3b: Memory Safety Vulnerability
(i) **Vulnerable Line:** Line 13 (`strcpy(entry->title, title);`)
(ii) **Reason:** `strcpy` does not enforce bounds checking on target buffer `header[8]`, allowing buffer overflow exploits.

Question 3c: Security Attack Method
Replay Attack.

Fall 2025 — Set A

Question 1a: Directory Navigation Storage
Directory blocks store entry mappings of file names to inode numbers. They contain special entries `.` (pointing to the current directory inode) and `..` (pointing to the parent directory inode), enabling `cd ..` to resolve the parent inode directly.

Question 1b: Uncommitted Journal Recovery
(i) **Issue:** Transaction `id=3` lacks a TxE completion block, rendering it incomplete.
(ii) **Prevention:** The recovery subsystem aborts and rolls back uncommitted transactions lacking TxE markers.

Question 1c: Triple Indirect Block Disk Reads

- Block size = 16 B, Pointer size = 4 B $\implies$ 4 pointers/block.
- Direct limit = $7 \times 16 = 112\ \text{Bytes}$
- Single indirect limit = $4 \times 16 = 64\ \text{Bytes}$ (112–175 B)
- Double indirect limit = $3 \times (4^2 \times 16) = 768\ \text{Bytes}$ (176–943 B)
- Byte 2880 falls in the Triple Indirect Block range.
- **Required Reads:** Inode $\rightarrow$ Triple Indirect Block $\rightarrow$ Double Indirect Block $\rightarrow$ Single Indirect Block $\rightarrow$ Target Data Block (5 Disk Reads).

Question 1d: Inode Sector Address Calculation

- Start Byte = $5 \times 2048 + (14 - 1) \times 512 = 10,240 + 6656 = 16,896\ \text{Bytes}$
- Sector Address = $\frac{16,896}{2048} = 8$

(i) **Byte Address:** 16,896 Bytes
(ii) **Sector Address:** Sector 8

Question 1e: Journal Space Bottlenecks
A constrained journal space causes frequent checkpointing, forcing pending writes to flush to disk continuously and slowing overall performance.

Question 2a: Paging and External Fragmentation
Paging breaks memory into fixed-size frames and assigns non-contiguous pages into available frames, eliminating external fragmentation.

Question 2b: Page Table Size Calculation

- Pages = $\frac{2^{32}}{2^{12}} = 2^{20}\ \text{pages}$
- Physical Frames (4 GB RAM) = $\frac{2^{32}}{2^{12}} = 2^{20}\ \text{frames} \implies 20\ \text{bits}$
- PTE size = $20 + 3 + 3 = 26\ \text{bits} \implies 4\ \text{Bytes}$
- Page Table Size = $2^{20} \times 4\ \text{Bytes} = 4\ \text{MB}$

Question 2c: Copy-On-Write Page Bounds

- Array size = $1200 \times 4\ \text{Bytes} = 4800\ \text{Bytes}$. Page size = 4096 Bytes $\implies$ 2 pages.
  (i) **Minimum newly created pages:** 2 pages
  (ii) **Maximum newly created pages:** 4 pages

Question 2d: TLB Miss Access Time

- $\text{EAT} = (\text{TLB} + m) \cdot \text{hit} + (\text{TLB} + 2m) \cdot \text{miss}$
- $247.405 = (\text{TLB} + 230)(0.9765) + (\text{TLB} + 460)(0.0235)$
- $247.405 = \text{TLB} + 235.405 \implies \text{TLB} = 12\ \text{ns}$

TLB access time: 12 ns

Question 2e: Non-Contiguous Allocation Validation
Yes, the process can be created because single-level paging maps non-contiguous free frames across RAM.

Question 2f: 3-Level Address Translation

Case i: Logical Address 51005

- Binary (16-bit): `1100 0111 0011 1101`
- Indices: $P_1 = 12$, $P_2 = 7$, $P_3 = 3$, Offset $d = 13$
- Translation: 2nd Outer PMT[12] $\rightarrow$ Frame 24 $\rightarrow$ Outer PMT[7] $\rightarrow$ Frame 18 $\rightarrow$ Inner PMT[3] $\rightarrow$ Frame 60.
- Physical Address = $60 \times 16 + 13 = 960 + 13 = 973$

Case ii: Logical Address 32689

- Binary (16-bit): `0111 1111 1011 0001`
- Indices: $P_1 = 7$, $P_2 = 15$, $P_3 = 11$, Offset $d = 1$
- Translation: 2nd Outer PMT[7] $\rightarrow$ Frame 12 $\rightarrow$ Outer PMT[15] $\rightarrow$ Invalid bit.
- Physical Address: **invalid**

Question 3a: Static Access Control Solution
Implement dynamic domain switching or access rights inheritance to elevate privileges dynamically.

Question 3b: Buffer Overflow Analysis
(i) **Vulnerable Line:** Line 13 (`strcpy(entry->title, title);`)
(ii) **Reason:** `strcpy` copies strings without verifying bounds against `title[8]`, opening a memory safety vulnerability.

Question 3c: Network Security Violation
Session Hijacking (or Network Sniffing / Session Reuse).